Lab 4: Support Vector Machines

Kernels, the margin, and what the multipliers tell you

DS5006 — Machine Learning

Fall 2026

Abstract

You are given a solver for the SVM dual. You are not given a classifier. This lab is about everything in between: turning a vector of Lagrange multipliers into a decision function, working out which training points actually determine it, and reading the two hyperparameters — the penalty C and the Gaussian kernel width s^2 — off the behaviour of the fitted model rather than off a textbook. There are three scientific questions. Each one is answered by running a pre-specified experiment and reporting what the numbers say; none of them has an answer you can look up, because each depends on your own data.

1 What you are given and what you write

Provided (`svmlab_provided.py` — do not modify).

- `make_dataset(roll_number)` — your personal training and test sets. The seed is derived from your roll number, so your data are yours alone and are identical on every machine you run on.
- `solve_dual(K, y, C)` — the maximiser $\boldsymbol{\alpha}$ of the dual objective of Section 6.5 of the lecture notes, subject to $\sum_i \alpha_i y_i = 0$ and $0 \le \alpha_i \le C$. It wraps `cvxopt`'s quadratic programming routine.
- The experiment grids `C_GRID` and `S2_GRID`, the regime threshold `TOL`, and the defaults `C_DEFAULT` and `S2_DEFAULT`. Use these exactly; do not invent your own.

Read the source of `solve_dual` before you start. Mapping the dual of the lecture notes onto the standard form

$$\min_{\mathbf{x}} \ \tfrac{1}{2}\mathbf{x}^\top P\mathbf{x} + q^\top \mathbf{x} \qquad \text{subject to} \qquad G\mathbf{x} \le h, \quad A\mathbf{x} = b$$

that a QP library expects is a skill worth having, and the comments walk through it: the sign flip that turns a maximisation into a minimisation, the two stacked blocks of G that express $0 \le \alpha_i \le C$ as one-sided inequalities, and the single row of A carrying $\sum_i \alpha_i y_i = 0$.

You write (`svm_lab.py`). Seven functions and three experiments. The functions are specified in the file's docstrings and summarised in Table 1.

Function	Returns	Marks
`linear_kernel(A, B)`	(n_A, n_B) Gram matrix	3
`rbf_kernel(A, B, s2)`	(n_A, n_B) Gram matrix	7
`classify_regimes(alpha, C, tol)`	$(n,)$ integer codes	6
`recover_intercept(alpha, y, K, C, tol)`	θ_0	10
`decision_values(alpha, theta0, y, K_test)`	$(n_{\text{test}},)$	4
`margin_width(alpha, y, K)`	$2/\lVert\boldsymbol{\theta}\rVert$	5
`kkt_residual(alpha, theta0, y, K, C, tol)`	a single float	5

Table 1: Part A. Forty marks for machinery, before any science.

Libraries. NumPy only, plus the provided module. You may use `matplotlib` for your report figures, but not inside the seven functions. Importing `scikit-learn`, `scipy`, `cvxopt` directly, or any other learning or optimisation library is an automatic zero for the affected section — the grader scans for it.

Install. `pip install cvxopt`, then run `python selftest_lab4.py`. Do this on day one, not the night before the deadline.

2 Conventions

These are graded exactly as written. They are not arbitrary — each one is a place where two reasonable people would choose differently, and the grader cannot read your mind.

Pinned conventions

1. The Gaussian kernel is parameterised by s^2, *not* by γ: $K(\mathbf{x}, \mathbf{x}') = \exp\big(-\|\mathbf{x} - \mathbf{x}'\|^2/(2s^2)\big)$.
2. Both kernels take `(A, B)` and return shape (n_A, n_B). Hence `K_test` is $(n_{\text{test}}, n_{\text{train}})$, in that order.
3. Regime codes are the integers 0 = not a support vector, 1 = free support vector, 2 = bounded support vector.
4. The regime tests are strict below and inclusive above:

$$\text{non-SV} : \alpha_i \le \texttt{tol}, \qquad \text{free} : \texttt{tol} < \alpha_i < C - \texttt{tol}, \qquad \text{bounded} : \alpha_i \ge C - \texttt{tol}.$$

5. `margin_width` returns $2/\|\boldsymbol{\theta}\|$, not $1/\|\boldsymbol{\theta}\|$, and `float('inf')` when $\boldsymbol{\theta} = \mathbf{0}$.
6. `recover_intercept` averages over *free* support vectors only, falling back to the construction of Section 3 when there are none.
7. `kkt_residual` returns `0.0` when there are no free support vectors.

The one that catches people

Bounded support vectors satisfy $y_i g(\mathbf{x}_i) \le 1$, not $y_i g(\mathbf{x}_i) = 1$. Including them when you average for θ_0 gives a number that looks entirely plausible and is wrong. Experiment 1 is partly designed to make that visible, so if you get it wrong you will see it.

3 The intercept when there are no free support vectors

For small C every support vector can be bounded, and then no training point satisfies $y_i g(\mathbf{x}_i) = 1$. The KKT conditions no longer pin θ_0 to a point. They still constrain it.

Write $f_i = \sum_j \alpha_j y_j K_{ij}$ for the discriminant without the intercept, so that $g(\mathbf{x}_i) = f_i + \theta_0$. The conditions of Proposition 6.1 of the lecture notes say

$$\alpha_i = 0 \Rightarrow y_i(f_i + \theta_0) \ge 1, \qquad \alpha_i = C \Rightarrow y_i(f_i + \theta_0) \le 1.$$

Multiplying through by $y_i \in \{-1, +1\}$ and using $y_i^2 = 1$, each point supplies either a lower or an upper bound on θ_0, and every one of them has the same form $y_i - f_i$. Collecting:

$$L = \max_{i \in I_{\text{lo}}} \big(y_i - f_i\big), \qquad U = \min_{i \in I_{\text{up}}} \big(y_i - f_i\big),$$

$$I_{\text{lo}} = \{i : y_i = +1,\ \alpha_i < C\} \cup \{i : y_i = -1,\ \alpha_i > 0\},$$
$$I_{\text{up}} = \{i : y_i = +1,\ \alpha_i > 0\} \cup \{i : y_i = -1,\ \alpha_i < C\}.$$

Any $\theta_0 \in [L, U]$ is consistent with the KKT conditions. **Return the midpoint** $(L + U)/2$**.** Work out for yourself why a free support vector collapses the interval to a point, and check that your general code reproduces the free-support-vector answer when free support vectors exist.

4 Scientific question 1: which points determine the hyperplane?

Fit once at $C =$ `C_DEFAULT`, $s^2 =$ `S2_DEFAULT`. Implement `experiment_1` to produce four results.

(a) The three regimes are real. Partition the training set by multiplier and, for each regime, report the range of $y_i g(\mathbf{x}_i)$ over its members. Proposition 6.1 predicts $= 1$, ≤ 1 and ≥ 1 respectively. Count the violations with a slack of 10^{-6}.

(b) $\boldsymbol{\theta}$ is built from the examples. Refit with the *linear* kernel at the same C. Now $\boldsymbol{\theta}$ is a vector in $\mathbb{R}^2$ you can actually form: $\boldsymbol{\theta} = \sum_i \alpha_i y_i \mathbf{x}_i$ (Equation 22 of the notes). Check it against your kernel-only code two ways — that $2/\|\boldsymbol{\theta}\|$ equals `margin_width`, and that $\mathbf{X}\boldsymbol{\theta} + \theta_0$ equals `decision_values`. Both should agree to roughly 10^{-14}. This is the one place in the lab where you can see the feature-space object directly; for the Gaussian kernel it does not exist as a finite vector.

(c) A wrong intercept that looks right. Recompute θ_0 the wrong way — averaging $y_i - f_i$ over *all* support vectors — and report `kkt_residual` under each. Report both the two intercepts and the two residuals.

(d) Most of your data is doing nothing. Delete every non-support-vector from the training set, refit at the same C and s^2, and compare the two discriminants on the *test* set: the largest absolute difference in $g(\mathbf{x})$, and the fraction of test points on which the predicted labels agree.

To discuss in your report. How much of the training set survived (d), and what does the result say about where the information in a fitted SVM is stored? In (c), the two intercepts differ by a modest amount — does the difference in *classification accuracy* on the test set match the difference in KKT residual? If not, which of the two numbers would you trust to tell you the intercept is wrong, and why?

5 Scientific question 2: what does C buy?

Sweep C over `C_GRID` at $s^2 =$ `S2_DEFAULT`, recording at each C: the margin width, the support-vector fraction, the counts of free and bounded support vectors, and the training and test error.

To discuss in your report. Over what range of C does the test error improve, and over what range does the margin width keep changing? They are not the same range, and the gap between them is the point of this question. State the factor by which the margin width changes across the grid and the range over which the test error is, for practical purposes, flat. What does that say about choosing C by validation — is there a single best value, or a range you cannot distinguish?

Look also at the composition of the support set. At the smallest C on the grid, how many free support vectors are there? Explain the number using the box constraint $0 \leq \alpha_i \leq C$ and the equality constraint $\sum_i \alpha_i y_i = 0$, and say what it implies for the intercept you computed in Section 3.

6 Scientific question 3: the two degenerate limits of the Gaussian kernel

Sweep s^2 over `S2_GRID` at $C =$ `C_DEFAULT`, recording at each s^2: the support-vector count, the free-support-vector count, the training and test error, and two properties of the training Gram matrix $\mathbf{K}$ —

- its condition number $\lambda_{\max}(\mathbf{K})/\lambda_{\min}(\mathbf{K})$;
- the fraction of the total eigenvalue mass carried by the single largest eigenvalue, $\lambda_{\max}/\sum_k \lambda_k$.

Clip tiny negative eigenvalues to zero before computing the second quantity; they are rounding error, not information. Use `numpy.linalg.eigvalsh`, which assumes symmetry, rather than `eig`.

To discuss in your report. Both ends of the grid are degenerate and both drive the support-vector count to its maximum. They are not the same failure. Use the two spectral statistics to say what $\mathbf{K}$ converges to at each end, and explain in one sentence each why that limit makes every training point a support vector. Then connect it to the errors: one end has near-zero training error and useless test error, the other has both errors equal and large. Which is which, and why does the training error alone fail to tell you that anything is wrong at either end?

Report the s^2 at which your test error is smallest, and say how far either side of it you could move before the test error rises by more than two percentage points.

7 What to submit

Upload a single zip file to Moodle named **exactly** with your roll number — `CS24M001.zip` — containing:

1. `svm_lab.py` — your completed code, with `ROLL_NUMBER` set to your roll number.
2. `results.json` — produced by running `python svm_lab.py`. The schema is fixed; the keys the grader reads are listed in Table 2.
3. `report.pdf` — at most four pages, including figures. Answer the "to discuss" paragraph of each scientific question. Lead with the number, then the explanation. A plot that is not referred to in the text earns nothing.

Do not include `svmlab_provided.py`; it is replaced with a clean copy when your work is graded. Do not submit a notebook.

8 Marks

Part A —	the seven functions	40
Part B —	experiment 1 (SQ1)	18
	experiment 2 (SQ2)	12
	experiment 3 (SQ3)	15
Report		15
Total		**100**

Part A is marked by hidden tests on data you have not seen, and it is the accessible floor: the seven functions are short, and `selftest_lab4.py` checks every convention they depend on. Part B re-runs your `experiment_*` functions rather than reading your `results.json`, so a hand-edited results file earns nothing.

Experiment	Key	Type
1	`regime_counts`	$[n_{\text{non}}, n_{\text{free}}, n_{\text{bounded}}]$
1	`free_margin_min`, `free_margin_max`	float
1	`bounded_margin_max`, `nonsv_margin_min`	float
1	`violations`	three ints, one per regime
1	`theta_width_diff`, `theta_g_diff`	float
1	`theta0_free`, `theta0_all_sv`, `theta0_abs_diff`	float
1	`kkt_residual_free`, `kkt_residual_all_sv`	float
1	`refit_n_before`, `refit_n_after`	int
1	`refit_max_g_diff`, `refit_label_agreement`	float
2	`C_grid`	list, same order as `C_GRID`
2	`margin_width`, `sv_fraction`	list of float
2	`n_free`, `n_bounded`	list of int
2	`train_error`, `test_error`	list of float
2	`width_ratio`, `test_error_improvement`	float
3	`s2_grid`	list, same order as `S2_GRID`
3	`n_sv`, `n_free`	list of int
3	`train_error`, `test_error`	list of float
3	`cond_K`, `top_eig_fraction`	list of float
3	`best_s2`, `test_error_left_rise`, `test_error_right_rise`	float

Table 2: The `results.json` schema. Lists must be in grid order.

Run the self-test before submitting. It takes a few seconds and it checks the conventions, which is where marks are most often lost for reasons that have nothing to do with understanding. Passing it is necessary, not sufficient.

Expected runtime. The full set of three experiments is about 80 quadratic programmes at $n = 240$, and should take well under a minute. If yours takes materially longer, you have a Python loop where a NumPy expression belongs — most often inside `rbf_kernel`, which is called with a 2000×240 argument.

Academic integrity

The usual course policy applies, and two things about this lab are worth stating explicitly.

Your data are seeded from your roll number, so every number in your `results.json` is yours. Two submissions reporting the same regime counts, the same margin widths or the same best s^2 did not arise independently.

If you use an AI tool, say so in your report: which tool, and what you asked it. Using one is not a violation; concealing it is. Be aware that the conventions in Section 2 are specific to this lab, and that a tool which has not been told them will produce code that fails the self-test in ways that are easy to recognise — a γ-parameterised kernel, an intercept averaged over every support vector, a margin of $1/\|\boldsymbol{\theta}\|$. You are responsible for every line you submit, including the ones you did not write, and you may be asked to explain any of them.